

Model Identification and Data Analysis 2

L^AT_EX by Min Soo Jeon
mjeon@uc.cl

Professor Sergio Savaresi
Politecnico di Milano

Semester 2025-1

Disclaimer:

This material is an unofficial resource created for educational purposes only. The authors and contributors do not assume any liability for errors, inaccuracies, or misinterpretations that may arise from its use. Readers should verify information from official sources.

Version 3.0.0

Permission is granted to reproduce this material, in whole or in part, for academic purposes, by any means, provided proper credit is given to the work and its author.

The commercialization of this material is strictly prohibited.

Introduction

These notes are based on the lectures delivered by Professor Sergio Savaresi during the Model Identification and Data Analysis 2 course at Politecnico di Milano during Semester 2025-1. Official course information can be found on the official website (Savaresi, 2025). Some sections, particularly the practical components, were taught by Teaching Assistant Giacomo Dolcaro and Jose Mendoza. Further details on the practical part can be found in the provided resources. This material was prepared independently by the author(s) and has not been reviewed or approved by the professors named herein. This material may be outdated, the latest version is available in the repository. To report an error, please contact mjeon@uc.cl.

Contents

1	System Representations	6
1.1	Generic System Control	6
1.2	State Space (SS) Representation	6
1.3	Transfer Function (TF) Representation	8
1.4	Impulse Response Representation	8
2	Representation Transformations	9
2.1	SS to TF	9
2.2	TF to SS	10
2.3	TF to IR	11
2.4	IR to TF	12
2.5	SS to IR	12
2.6	IR to SS	13
2.6.1	Fully Observability	13
2.6.2	Fully Controllability	14
2.6.3	4 Subsystems of SS representation	15
2.6.4	Hankel Matrix	15
2.6.5	4SID Noise-free Algorithm	16
2.6.6	4SID Noisy	17
3	Kalman Filter (KF)	19
3.1	Introduction	19
3.2	SW sensing problem	20
3.3	Algorithm Introduction	21
3.4	Solution for 1-step prediction	22
3.5	Extensions	23
3.6	Linear Time Variant	24
3.7	Stability and existence for convergent systems	25
3.8	Non-linear Systems	27
4	Minimum Variance Control (MVC)	30
4.1	Introduction	30
4.2	The problem	31

4.3	Optimal Predictor	32
4.4	Stability	33
4.5	y estimation	34
4.6	Choosing predictor for MVC	34
4.7	Generalized MVC (GMVC)	35
4.8	Discretization of Analog Dynamical Systems	36
4.8.1	State-Space Discretization	36
4.8.2	Derivative Discretization	37
4.8.3	Choosing ΔT	37
5	Frequency Domain Model Identification	39
5.1	Introduction	39
5.2	Method Idea	39
5.3	System Excitation	39
5.4	Denoised Measures	41
5.5	Performance Index Definition	42
5.6	Additional Remarks	42
6	Software Sensing (SW) with B.B methods	43
6.1	Introduction	43
6.2	Recurrent (Dynamical) Neural Networks	44
6.3	FIR architecture	44
6.4	IIR architecture	45
6.5	Meta Architecture	45
6.6	Model Based vs Black-box SW sensing	45
7	Grey Box (GB) System Identification	45
7.1	Introduction	45
7.2	Kalman Filter with State Extension	46
7.3	Optimization	47
	References	48

1 System Representations

1.1 Generic System Control

The typical architecture of a generic control system consists of an input $u(t)$, that goes through a system and gives an output $y(t)$. Then, by SW-sensing (Software Sensing) algorithms, we can estimate a $\hat{y}(t)$ through the physical measurable variables. Finally, we can use control algorithms to evaluate the estimator.

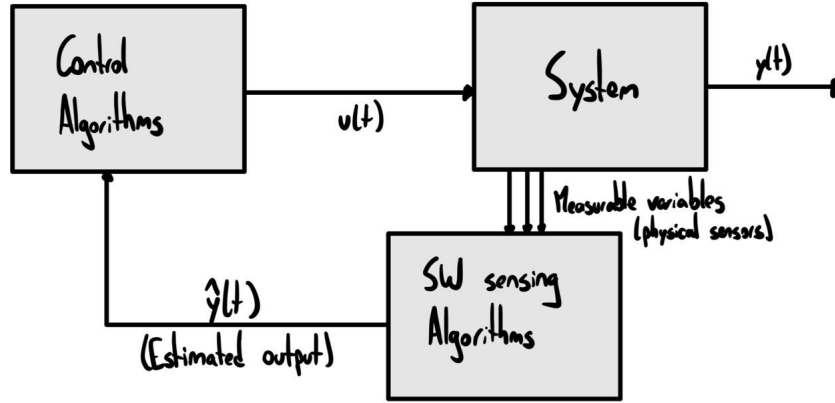


Figure 1: Generic System Control Representation

1.2 State Space (SS) Representation

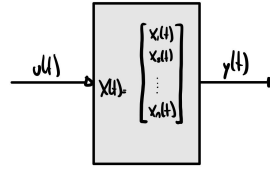


Figure 2: State Space Representation

Where $x(t)$ is a vector of internal variables called **States**.

$$\begin{cases} x(t+1) = Fx(t) + Gu(t) & \text{State Equation} \\ y(t) = Hx(t) + Du(t) & \text{Output Equation} \end{cases}$$

$$F \in \mathbb{R}^{n \times n} \quad \wedge \quad G \in \mathbb{R}^{n \times k} \quad \wedge \quad H \in \mathbb{R}^{m \times n} \quad \wedge \quad D \in \mathbb{R}^{m \times k}$$

Where n is the order of the system, m is the output size, and k is the input size.

Definition 1.1: Strictly Proper System

D is called a Strictly Proper System if:

$$D = [0]$$

i.e. $u(t)$ does not affect $y(t)$ directly

When a system is strictly Proper, the jumps in the input $u(t)$ give continuous asymptotic responses, rather than discontinuous behaviors copying the input.

Anyway, in the discrete setup, this will be represented as discrete jumps, but following the continuous curve

Definition 1.2: SISO

Single In Single out: Just one input and output.

Remark.

Stability:

- Continuous Time Domain:

$$\begin{cases} \dot{x} = Ax(t) + Bu(t) & \text{State Equation} \\ y(t) = Cx(t) + Du(t) & \text{Output Equation} \end{cases}$$

In this case, the system is asymptotically stable if the eigenvalues are in the negative real part. The system is simple stable if they are in the neutral real part. The system is unstable otherwise.

- Discrete Time Domain:

$$\begin{cases} x(t+1) = Fx(t) + Gu(t) & \text{State Equation} \\ y(t) = Hx(t) + Du(t) & \text{Output Equation} \end{cases}$$

In this case, the system is asymptotically stable if the eigenvalues are in the unitary circle in the complex plane. The system is simple stable if they are in the unitary circumference in the complex plane. The system is unstable otherwise.

Example.

$$\begin{cases} x_1(t+1) = \frac{1}{2}x_1(t) + 2u(t) \\ x_2(t+1) = x_1(t) + 2x_2(t) + u(t) \\ y(t) = \frac{1}{4}x_1(t) + \frac{1}{2}x_2 \end{cases}$$

$$F = \begin{bmatrix} 1/2 & 0 \\ 1 & 2 \end{bmatrix} \quad \wedge \quad G = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \quad \wedge \quad H = \begin{bmatrix} 1/4 & 1/2 \end{bmatrix} \quad \wedge \quad D = \begin{bmatrix} 0 \end{bmatrix}$$

The system is out of the unit circle, so it is unstable. Plus, the system is strictly proper.

Remark.

State Space Representation is not unique:

Given any T_1 square and invertible matrix, we can take:

$$F_1 = T_1 F T_1^{-1} \quad \wedge \quad G_1 = T_1 G \quad \wedge \quad H_1 = H T_1^{-1} \quad \wedge \quad D_1 = D$$

And the representation is equivalent.

1.3 Transfer Function (TF) Representation

The Transfer Function Representation can be obtained from a time domain difference equation of the system, using the decay operator z^{-1} (See section 4.5 at (Jeon, 2025)). It is also called external representation, since the internal set is not represented.

Example.

Let be the following I/O difference equation

$$y(t) = -\frac{1}{6}y(t-1) - \frac{1}{8}y(t-2) + \frac{1}{2}u(t-1) + \frac{1}{4}u(t-2)$$

Using decay operator:

$$y(t) = -\frac{1}{6}z^{-1}y(t) - \frac{1}{8}z^{-2}y(t) + \frac{1}{2}z^{-1}u(t) + \frac{1}{4}z^{-2}u(t)$$

So, finally:

$$y(t) = \frac{\frac{1}{2}z^{-1} + \frac{1}{4}z^{-2}}{1 + \frac{1}{6}z^{-1} + \frac{1}{8}z^{-2}}u(t) = W(z)u(t)$$

$W(z)$ is called Transfer Function from input $u(t)$ to output $y(t)$.

In general, we can take the representation:

$$W(z) = \frac{B(z)}{A(z)}z^{-k} = \frac{b_0 + b_1z^{-1} + b_2z^{-2} + \dots + b_pz^{-p}}{a_0 + a_1z^{-1} + a_2z^{-2} + \dots + a_nz^{-n}}z^{-k}$$

Remark.

If the system is strictly proper:

$$\implies k \geq 1$$

1.4 Impulse Response Representation

The convolution of the input with the impulse response consists of a unique representation given by the output of the system when there is an output with a 1 in $t=0$ and 0 in the rest

of the periods.

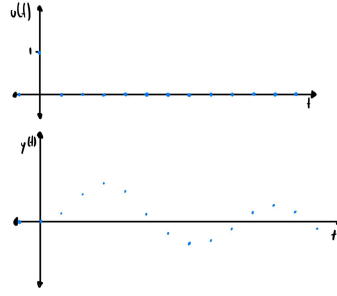


Figure 3: Impulse Response Representation Example

The IR representation is unique, where the outputs are represented with $\omega(i)$:

$$IR = \{\omega(0), \omega(1), \omega(2), \dots\}$$

It can be proven that the output of the system can be written as:

$$y(t) = \sum_{k=0}^{+\infty} \omega(k)u(t-k)$$

2 Representation Transformations

2.1 SS to TF

The most classical Transformation is from State Space to Transform Function Representations:

$$\begin{cases} x(t+1) = Fx(t) + Gu(t) \\ y(t) = Hx(t) \end{cases}$$

(Considering an Strictly Proper System.)

Using the z operator in the first equation:

$$\begin{aligned} \implies zx(t) &= Fx(t) + Gu(t) \\ \implies (zI - F) \cdot x(t) &= Gu(t) \quad (\text{I Identity Matrix}) \end{aligned}$$

So replacing in the second equation:

$$y(t) = H(zI - F)^{-1}Gu(t)$$

Similarly, if the system is not Strictly Proper:

$$y(t) = H(zI - F)^{-1}Gu(t) + D$$

Example.

$$\begin{cases} x_1(t+1) = x_1(t) + u(t) \\ x_2(t+1) = \frac{1}{2}x_1(t) + 2x_2(t) + u(t) \\ y(t) = x_1(t) \end{cases}$$

This is a SISO with two states:

$$F = \begin{bmatrix} 1 & 0 \\ 1/2 & 2 \end{bmatrix} \quad \wedge \quad G = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad \wedge \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix} \quad \wedge \quad D = \begin{bmatrix} 0 \end{bmatrix}$$

Then:

$$\begin{aligned} W(z) &= \begin{bmatrix} 1 & 0 \end{bmatrix} \cdot \left(\begin{bmatrix} z & 0 \\ 0 & z \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ 1/2 & 2 \end{bmatrix} \right)^{-1} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 1/2 & 2 \end{bmatrix} \begin{bmatrix} z-1 & 0 \\ -1/2 & z-2 \end{bmatrix}^{-1} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 1/2 & 2 \end{bmatrix} \frac{1}{(z-1)(z-2)} \begin{bmatrix} z-2 & 0 \\ 1/2 & z-1 \end{bmatrix}^{-1} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \frac{z-2}{(z-2)(z-1)} = \frac{1}{z-1} \\ &= \frac{z^{-1}}{1-z^{-1}} \end{aligned}$$

An alternative way is to simply isolate the variables and replace in the equations.

Note that the SS representation has order $n=2$, so there are two eigenvalues of F .

TF representation, instead, is a system of order 1 (1 pole). This mismatch of orders means the system is not fully observable

2.2 TF to SS

The transformation from Transfer Function to Steady State is not very commonly used in practice. This transformation is called “Realization” of a TF into a SS model.

The problem with this representation is that SS representation is not unique.

The next formula of a special realization is called “control realization”:

Let be the pole monic, and a system strictly proper

$$W(z) = \frac{b_0 z^{n-1} + b_1 z^{n-2} + \dots + b_{n-1}}{1 \cdot z^n + a_1 z^{n-1} + a_2 z^{n-2} + \dots + a_n} + d$$

So a realization of the SS representation is:

- F has a 0 in the diagonal and 1 in the superdiagonal, and 0 in the rest of the cells.
This, except for the last row, where the elements are: $\begin{pmatrix} -a_n & -a_{n-1} & \dots & -a_1 \end{pmatrix}$
- G is a vector of only 0 and a 1 in the last row

- $H = \begin{bmatrix} b_{n-1} & b_{n-2} & \dots & b_1 & b_0 \end{bmatrix}$
- $D = d$

Example.

Let be

$$W(z) = \frac{2z^2 + \frac{1}{2}z^1 + \frac{1}{4}}{z^3 + \frac{1}{4}z^2 + \frac{1}{3}z + \frac{1}{5}}$$

Then, the matrices are:

•

$$F = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -\frac{1}{5} & -\frac{1}{3} & -\frac{1}{4} \end{bmatrix}$$

•

$$G = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

•

$$H = \begin{bmatrix} \frac{1}{4} & \frac{1}{2} & 2 \end{bmatrix}$$

• $D = 0$

2.3 TF to IR

The transformation from Transfer Function to Impulse Response consists of simply applying long polynomial division between $B(z)$ and $A(z)$.

Example.

Let be

$$W(z) = \frac{1}{z - \frac{1}{2}}$$

Then, by long division:

$$W(z) = 0 \cdot z^0 + 1 \cdot z^{-1} + \frac{1}{2} \cdot z^{-2} + \frac{1}{4} \cdot z^{-3} + \frac{1}{8} \cdot z^{-4} + \frac{1}{16} \cdot z^{-5}$$

So:

$$IR = \{0, 1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \dots\}$$

On another point of view, note that:

$$y(t) = z^{-1} \cdot \frac{1}{1 - \frac{1}{2}z^{-1}} u(t)$$

By the geometrical series:

$$y(t) = z^{-1} \cdot \sum_{k=0}^{\infty} \left(\frac{1}{2}z^{-1}\right)^k \cdot u(t)$$

And we can get to the same result.

Remark.

- When we have $A(z) \neq 1$, $W(z)$ is called infinite impulse response (IIR), since the IR will be infinite. This, happens due to the recursive part.
- When we have $A(z) = 1$, $W(z)$ is called finite impulse response (FIR), since the IR will be “finite” (there will be just a finite number of elements different from 0).

2.4 IR to TF

The transformation from Impulse Response to Transfer Function needs to use the geometric series.

Definition 2.1: Z-transform

Given a discrete-time signal $s(t)$ with $s(t) \neq 0$. If $t < 0$, the z transformation of signal $s(t)$ can be represented as the z-transform of signal $s(t)$:

$$Z(s(t)) = \sum_{t=0}^{\infty} s(t)z^{-t}$$

It can be proven that:

$$W(z) = Z(\omega(t)) = \sum_{t=0}^{\infty} \omega(t)z^{-t}$$

So the Z-transform of the IR is the transfer function of a system.

Anyway, this way of calculation can lead to the following problems:

- Infinite values of IR are needed.
- IR signal must be noise-free available.

2.5 SS to IR

The transformation from State Space to Impulse Response is done as the following example:

Example.

$$\begin{cases} x(t+1) = Fx(t) + Gu(t) \\ y(t) = Hx(t) \\ x(0) = 0 \quad \wedge \quad y(0) = 0 \end{cases}$$

Consider Strictly Proper System.

The system can be simply “run”, starting from the initial conditions:

1. $x(1) = Fz(0) + Gu(0) = Gu(0)$ and $y(1) = Hx(1) = HGu(0)$
2. $x(2) = Fz(1) + Gu(1) = FGu(0) + Gu(1)$ and $y(2) = Hx(2) = HFGu(0) + HGu(1)$
3. ...
- 4.

$$y(t) = 0u(t) + HGu(t-1) + HFGu(t-2) + HF^2Gu(t-3) + \dots$$

So:

$$W(t) = \begin{cases} 0 & \text{if } t = 0 \\ HF^{t-1}G & \text{if } t > 0 \end{cases}$$

So we have a function from $\{H, F, G\}$ to $\mathbb{R}$.

2.6 IR to SS

The transformation from Impulse Response to State Space is the key task of a Black-Box system identification method called 4SID. 4SID stands for Subspace-based State Space System Identification Method.

Now, we need to recall the fundamental concepts of observability and controllability of a system. Given a SS representation of a Strictly Proper System:

$$\begin{cases} x(t+1) = Fx(t) + Gu(t) \\ y(t) = Hx(t) \end{cases}$$

2.6.1 Fully Observability

Definition 2.2: Observability Matrix

$$\mathcal{O} = \begin{bmatrix} H \\ HF \\ HF^2 \\ \dots \\ HF^{n-1} \end{bmatrix}$$

Definition 2.3: Fully Observable

A system is fully observable from the output $y(t)$ if the observability matrix is full rank, i.e.:

$$\text{rank}(\mathcal{O}) = n$$

Example.

Let be the following system:

$$\begin{cases} x_1(t+1) = \frac{1}{2}x_1(t) + u(t) \\ x_2(t+1) = \frac{1}{3}x_2(t) \\ y(t) = \frac{1}{4}x_1(t) \end{cases}$$

$$F = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/3 \end{bmatrix} \quad \wedge \quad G = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \wedge \quad H = \begin{bmatrix} 1/4 & 0 \end{bmatrix}$$

Then, the observability matrix is:

$$\mathcal{O} = \begin{bmatrix} 1/4 & 0 \\ 1/8 & 0 \end{bmatrix}$$

The rank of the matrix is $1 < 2$, so the system is not fully observable.

It can be noted that x_2 is not reflected in the output, so this makes the system not fully observable.

2.6.2 Fully Controllability

Definition 2.4: Controllability Matrix

$$\mathcal{R} = \begin{bmatrix} G & FG & F^2G & \dots & F^{n-1}G \end{bmatrix}$$

Definition 2.5: Fully Controllable

A system is fully controllable if the controllability matrix is full rank

Example.

Let be the following system:

$$\begin{cases} x_1(t+1) = \frac{1}{2}x_1(t) \\ x_2(t+1) = \frac{1}{3}x_2(t) + u(t) \\ y(t) = \frac{1}{4}x_1(t) \end{cases}$$

$$F = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/3 \end{bmatrix} \quad \wedge \quad G = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad \wedge \quad H = \begin{bmatrix} 1/4 & 0 \end{bmatrix}$$

Then, the controllability matrix is:

$$\mathcal{R} = \begin{bmatrix} 0 & 0 \\ 1 & 1/3 \end{bmatrix}$$

The rank of the matrix is $1 < 2$, so the system is not fully controllable.

It can be noted that $u(t)$ is not reflected in the output, so this makes the system not fully controllable.

2.6.3 4 Subsystems of SS representation

Each SS system can be divided into 4 subsystems:

- **Non Observable Controllable (NO C):** It is connected with u but not with y .
- **Observable non Controllable (O NC):** It is not connected with u but connected with y .
- **Observable Controllable (OC):** It is connected with u and y .
- **Non Observable non Controllable (NO NC):** It is not connected with u or y .

The external representation (TF) can describe only the CO part.

2.6.4 Hankel Matrix

Definition 2.6: Hankel Matrix

The Hankel Matrix of order n built from IR is:

$$H_n = \begin{bmatrix} \omega(1) & \omega(2) & \omega(3) & \dots & \omega(n) \\ \omega(2) & \omega(3) & \omega(4) & \dots & \omega(n+1) \\ \omega(3) & \omega(4) & \omega(5) & \dots & \omega(n+2) \\ \dots & & & & \\ \omega(n) & \omega(n+1) & \omega(n+2) & \dots & \omega(2n-1) \end{bmatrix} \in \mathbb{R}^{n \times n}$$

Remark.

Note that the cross diagonals are equal. Plus, for a order n matrix, we need information until $2n-1$.

Recall from section 2.5, that $\omega(t) = HF^{t-1}G$, so:

$$H_n = \begin{bmatrix} HG & HFG & HF^2G & \dots & HF^{n-1}G \\ HFG & HF^2G & HF^3G & \dots & HF^nG \\ HF^2G & HF^3G & HF^4G & \dots & HF^{n+1}G \\ \dots & & & & \\ HF^{n-1}G & HF^nG & HF^{n+1}G & \dots & HF^{2n-2}G \end{bmatrix} = \begin{bmatrix} H \\ HF \\ HF^2 \\ \dots \\ HF^{n-1} \end{bmatrix} \begin{bmatrix} G & FG & \dots & F^{n-1}G \end{bmatrix}$$

So definitively:

$$H_n = \mathcal{O} \cdot \mathcal{R}$$

2.6.5 4SID Noise-free Algorithm

With the 4SID algorithm, an estimated SS model can be obtained from experiments from an IR system.

1. Build the Hankel Matrices in increasing orders, and check the ranks. When the we find the first not full rank matrix, we stop generating matrix, and set the last full rank n as the order of the system.

In real life, we do not necessarily stop with this rule, sometimes the form of the impulse response or the availability of the data leads us to want to take more elements for the prediction, and consider them is perfectly good, and even better.

2. Take $H_{n+1} \in \mathbb{R}^{(n+1)(n+1)}$, matrix or order n . Factorize it into two rectangular matrices. In particular:

$$H_{n+1} = \mathcal{O}_{n+1} \cdot \mathcal{R}_{n+1}$$

- $\mathcal{O}_{n+1} \in \mathbb{R}^{(n+1) \times n}$
- $\mathcal{R}_{n+1} \in \mathbb{R}^{n \times (n+1)}$

3. Estimate F, G, and H from $\mathcal{O}_{n+1}$ and $\mathcal{R}_{n+1}$:

- $\hat{H}$ with the first element of $\mathcal{O}_{n+1} : \mathcal{O}_{n+1}[1; :]$
- $\hat{G}$ with the first element of $\mathcal{R}_{n+1} : \mathcal{R}_{n+1}[:, 1]$
- $\hat{F}$: Define $\mathcal{O}_1$ and $\mathcal{O}_2$ as $\mathcal{O}$ without the last and first element respectively.
As $\mathcal{O}_2 = \mathcal{O}_1 \cdot F$, we estimate:

$$\hat{F} = \mathcal{O}_1^{-1} \cdot \mathcal{O}_2$$

- $\hat{D} = \omega(0)$

Remark.

A general rule for factorizing H_{n+1} is to put n independent rows in $\mathcal{R}_{n+1} = H_{n+1}(1 : n, :)$. In $\mathcal{O}_{n+1}$, put in the first n columns the identity I_n and in the last one there are the coefficients corresponding to the linear combination of the last row in H_{n+1} (since it is not full rank).

Remark.

This method is non-parametric/constructive. A typical parametric M.L. algorithm has the following path:

1. Collecting a clean dataset:

$$\{(u(1), y(1)), \dots, (u(N), y(N))\}$$

2. Choose a parametric model $M(\theta) : y(t) = W(Z, \theta)u(t)$

3. Define a performance index:

$$J(\theta) = \frac{1}{N} \sum_{t=1}^N (y(t) - w(z; \theta)u(t))^2$$

This evaluate the quality of the model, it is said that if $J(\theta_1) < J(\theta_2)$, $M(\theta_1)$ is better than $M(\theta_2)$

4. The optimization step: Minimize $J(\theta)$:

$$\hat{\theta}_N = \arg \min_{\theta} J(\theta)$$

Choosing the model $M(\hat{\theta}_N)$

2.6.6 4SID Noisy

4SID noise-free data has been known since 60's.

During 90's, Professor Golub from Stanford, rediscovered it, when a fundamental tool of numerical algebra was fully developed: Singular Values Decomposition (SVD).

Now, assume the IR is measured with noise ($\hat{\omega}(t)$). It can be decomposed into noise-free IR and noise:

$$\hat{\omega}(t) = \omega(t) + \eta(t), \quad t = 0, 1, 2, \dots, N$$

Remark.

For simplicity, we will see the algorithm for an impulse response, but the analysis can be done with any type of input signal.

The algorithm is as follows:

1. Build the hankel matrix using the full dataset:

$$\tilde{H}_{qd} = \begin{bmatrix} \omega(1) & \omega(2) & \omega(3) & \dots & \omega(d) \\ \omega(2) & \omega(3) & \omega(4) & \dots & \omega(d+1) \\ \omega(3) & \omega(4) & \omega(5) & \dots & \omega(d+2) \\ \dots & & & & \\ \omega(q) & \omega(q+1) & \omega(q+2) & \dots & \omega(q+d-1) \end{bmatrix} \in \mathbb{R}^{q \times d}$$

q+d-1=N, i.e., the full dataset.

Assume that $q < d$, then $q = N - d + 1$. As q is closer to 1, the model quality is worse, but at a better computational saving. On the other side, as $q \approx d$, the model quality is better, but it needs more computational resources.

Typically, $q > \frac{1}{2}d$ is enough, with very similar results from this threshold, i.e., from this point, the sensitivity is low.

2. SVD of $\tilde{H}_{qd}$:

$$\tilde{H}_{qd} = \tilde{U} \tilde{S} \tilde{V}^T$$

Where $\tilde{U}, \tilde{V}$ are unitary matrices, i.e., determinant equal to 1 and the inverse equal to the transpose.

On the other side, $\tilde{S}$ is a diagonal rectangular matrix. The elements of its diagonal $\sigma_1, \sigma_2, \dots, \sigma_q$ are called “singular values” (SV) of $\tilde{H}_{qd}$. We notate them in decreasing order: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_q$

Remark.

SVD is a sort of diagonalization of a rectangular matrix. The singular values have a similar role as eigenvalues, but in a rectangular matrix. Actually:

$$SV(M) = \sqrt{EIG(MM^T)}$$

3. Plot the singular values and cut the system into noise and signal. In the ideal situation, there is a clear jump after a few SVs (order n) to the singular values corresponding to noise. In reality, this transition is not that clear.

With this, a separation in the system can be done. A typical example cuts a lot of SVs. For example, $N = 1000, q = 400, n = 8$.

The cut of the SVD matrices is performed as:

- $\hat{U}$: The first n columns of $\tilde{U}$
- $\hat{S}$: The first $n \times n$ submatrix of $\tilde{S}$
- $\hat{V}$: The first n rows of $\tilde{V}^T$

$$\hat{H}_{qd} = \hat{U} \hat{S} \hat{V}^T$$

4. Estimation of $\hat{F}, \hat{G}, \hat{H}$, starting from the clean matrix $\hat{H}_{qd}$. Actually, it can be proven:

$$\hat{\mathcal{O}} = \hat{U} \hat{S}^{1/2} \in \mathbb{R}^{q \times n}$$

$$\hat{\mathcal{R}} = \hat{S}^{1/2} \hat{V}^T \in \mathbb{R}^{n \times d}$$

These matrices can be seen as the extended observability and controllability matrices of the system, and:

$$\hat{H}_{qd} = \hat{\mathcal{O}} \cdot \hat{\mathcal{R}}$$

As we have seen:

$$\hat{H} = \hat{\mathcal{O}}(1; :) \quad \wedge \quad \hat{G} = \hat{\mathcal{R}}(:, 1)$$

And:

$$\mathcal{O}_1 = \hat{\mathcal{O}}(1 : q - 1; :) \quad \wedge \quad \mathcal{O}_2 = \hat{\mathcal{O}}(2 : q; :)$$

And finally, by the shift invariance property:

$$\mathcal{O}_1 F = \mathcal{O}_2$$

So (See remark below):

$$\hat{F} = (\mathcal{O}_1^T \mathcal{O}_1)^{-1} \mathcal{O}_1^T \mathcal{O}_2$$

Remark.

Solution of linear system $Ax=b$, $A \in \mathbb{R}^{h \times n}$, $x \in \mathbb{R}^{n \times 1}$, $b \in \mathbb{R}^{h \times 1}$:

- If $h < n$, there are fewer equations than variables and the system is underdetermined. Therefore, there are infinite solutions.
- If $h = n$, there is one and only one solution, unless A is not invertible.
- If $h > n$, there are more equations than variables, so the system is overdetermined, and there's no solution.

Anyway, this system can be solved approximately, with least squares:

$$\hat{x} = (A^T A)^{-1} \cdot A^T \cdot B$$

Remark.

The 4SID method is optimal since SVD makes an optimal rank reduction.

A rank reduction tries to find a matrix of rank $n \ll q$ such that the residual is minimum:

$$\underbrace{\tilde{H}_{qd}}_{rank=q} = \underbrace{\hat{H}_{qd}}_{rank=n} + \underbrace{H_{res,qd}}_{rank=q}$$

Strictly, we try to minimize $H_{res,qd}$ with respect to the Frobenius norm:

$$|H_{res,qd}|_F = \sqrt{\sum_{i,j} (H_{res,qd}^{i,j})^2}$$

SVD achieved this in an efficient way, being a turning point in a lot of algorithms using matrices.

3 Kalman Filter (KF)

3.1 Introduction

The Kalman Filter is a model-based software sensing (SW). The method is based on the SS representation of the system:

$$\begin{cases} x(t+1) = Fx(t) + Gu(t) + v_1(t) & \text{State Equation} \\ y(t) = Hx(t) + Du(t) + v_2(t) & \text{Output Equation} \end{cases}$$

where v_1 and v_2 are noises (non-measurable).

We will assume that the system model $\{F, G, H\}$ is given, typically built in a White-Box way.

Given a model of a system, i.e.,

$$\{u(1), u(2), \dots, u(N)\}$$

$$\{y(1), y(2), \dots, y(N)\}$$

With Kalman Filter we can solve the following problems:

- k-steps ahead prediction of the output, i.e., find $\hat{y}(N + k|N)$.
This problem can be solved with prediction theory as well, see (Jeon, 2025).
- k-steps prediction of state, i.e., find $\hat{x}(N + k|N)$.
This is a new problem, where the states are not directly measured.
- Filter of the state, i.e., find $\hat{x}(N|n)$, estimate the state at a present time.
- Gray Box system Identification (see chapter 5Missing). Although it is not the main objective of KF, it can be used as well.

3.2 SW sensing problem

The SW sensing consists of, given inputs:

$$\{u_1(t), u_2(t), \dots, u_1(m)\}$$

And outputs:

$$\{y_1(t), y_2(t), \dots, y_p(t)\}$$

Find internal states:

$$\{x_1(t), x_2(t), \dots, x_n(t)\}$$

The problem is that usually $n \gg p$, since the physical sensors are expensive.

Usually, only a few states are measured, but the availability of the full state is very useful for designing control algorithms and monitoring the system for fault detection and predictive maintenance.

Therefore, the problem can be solved by adding extra physical sensors or developing sw-sensing algorithms. In some cases, there are no physical sensors for what we want to measure, but in most cases both options are available. Under these circumstances, a balance between fixed and variable costs.

Usually, for high-volume production, SW-sensing is better, while for low-volume production, installing physical sensors is better.

In a lot of cases, we might want to use both, since we need redundancy for safety critical applications.

One problem with SW is that it is not always feasible. In these cases, we need to check the observability. Another problem with SW is the quality of the estimation. This must be low so it provides valuable data.

3.3 Algorithm Introduction

We will introduce KF algorithm starting from a basic system. It will be linear, time invariant, and without exogenous inputs. Then, the system will becomes:

$$\begin{cases} x(t+1) = Fx(t) + v_1(t) & \text{State Equation} \\ y(t) = Hx(t) + v_2(t) & \text{Output Equation} \end{cases}$$

$$v_1(t) = \begin{bmatrix} v_{11}(t) \\ v_{12}(t) \\ \dots \\ v_{1n}(t) \end{bmatrix} \sim WN(0, V_1) \text{ This is called state noise of model noise, and it is used to}$$

model internal disturbances or small model errors. It satisfies the following characteristics:

$$\bullet E[v_1(t)] = \underline{0} = \begin{bmatrix} 0 \\ 0 \\ \dots \\ 0 \end{bmatrix}$$

- $E[v_1(t) \cdot v_1(t)^T] = V_1$ is the covariance matrix of v_1 . It is a squared, symmetric, and semidefinite positive (sdp) matrix $V_1 \geq 0$.

$$\bullet E[v_1(t) \cdot v_1(t - \tau)^T] = \underline{0} \quad \forall t \forall \tau \neq 0$$

$$v_2(t) = \begin{bmatrix} v_{21}(t) \\ v_{22}(t) \\ \dots \\ v_{2l}(t) \end{bmatrix} \sim WN(0, V_2) \text{ This is called output sensor error, and it is used to model}$$

noise of physical sensors. it satisfies the following characteristics:

$$\bullet E[v_2(t)] = \underline{0}$$

- $E[v_2(t) \cdot v_2(t)^T] = V_2$ is the covariance matrix of v_2 . It is an sdp matrix $V_2 \geq 0$ as well, but we will request being d.p. ($V_2 > 0$).

$$\bullet E[v_2(t) \cdot v_2(t - \tau)^T] = \underline{0} \quad \forall t \forall \tau \neq 0$$

In addition, regarding the cross-correlation matrix, we assume they can be correlated only at the same time:

$$E[v_1(t) \cdot v_2(t - \tau)^T] = \begin{cases} 0 & \text{if } \tau \neq 0 \\ V_{12} & \text{if } \tau = 0 \end{cases}$$

Since the system is a dynamical system, we must set the initial condition $x(1)$:

$$E[x(1)] = \underline{x}_0$$

$$E[(x(1) - \underline{x}_0)(x(1) - \underline{x}_0)^T] = Var[(x_1 - x_0)] = \underline{P}_0$$

If $\underline{P}_0 = 0$, we know exactly the initial conditions.

Finally, we assume that both v_1 and v_2 are correlated with $x(1)$, i.e.:

$$x(1) \perp v_1(t) \quad \wedge \quad x(1) \perp v_2(t) \quad \forall t$$

This is a technical assumption that we won't be able to verify, but it is necessary for KF optimality.

3.4 Solution for 1-step prediction

The Kalman Filter algorithm consists of the following predictors:

$$\begin{aligned}
 \hat{x}(t+1|t) &= F\hat{x}(t|t-1) + K(t) \cdot e(t) && \text{State equation + "correction"} \\
 \hat{y}(t|t-1) &= H\hat{x}(t|t-1) && \text{Output equation} \\
 e(t) &= y(t) - \hat{y}(t|t-1) && \text{Error Equation} \\
 K(t) &= (FP(t)H^T + V_{12})(HP(t)H^T + V_2)^{-1} && \text{Gain Equation}
 \end{aligned}$$

Where $P(t)$ is the Difference Riccati Equation (DRE):

$$P(t+1) = (FP(t)F^T + V_1) - (FP(t)H^T + V_{12})(HP(t)H^T + V_2)^{-1}(FP(t)H^T + V_{12})^T$$

And the initial conditions are:

$$\hat{x}(1|0) = \underline{x}_0 \quad \wedge \quad P(1) = \underline{P}_0$$

Remark.

Note that $K(t)$ and DRE have a simple "block-structure". Define the blocks:

- State Block: F and V_1 are linked to the state equation:

$$FP(t)F^T + V_1$$

- Output Block: H and V_2 are linked to the output equation:

$$HP(t)H^T + V_2$$

- Mix Block: Mix between State and output:

$$FP(t)H^T + V_{12}$$

So note that:

$$\begin{aligned}
 K(t) &= (MIX) \cdot (OUTPUT)^{-1} \\
 P(t+1) &= (STATE) - (MIX)(OUTPUT)^{-1}(MIX)^T
 \end{aligned}$$

Remark.

DRE is a special type of non-linear matrix difference equation. It has no input, it is a recursive autonomous system.

Remark.

To guarantee the existence of DRE, $\forall t$ the matrix $HP(t)H^T + V_2$ must be invertible. Note that the first term is sdP, and V_2 we assured it dp. Then, overall, the matrix is definite positive.

Remark.

$P(t)$ is a $n \times n$ symmetric, sdP matrix, and it is: the covariance matrix of the 1-step error on the state:

$$P(t) = E[(x(t) - \hat{x}(t|t-1))(x(t) - \hat{x}(t|t-1))^T] = \text{Var}[(x(t) - \hat{x}(t|t-1))]$$

The system δ and Kalman Filter (KF) look something like the following diagram:

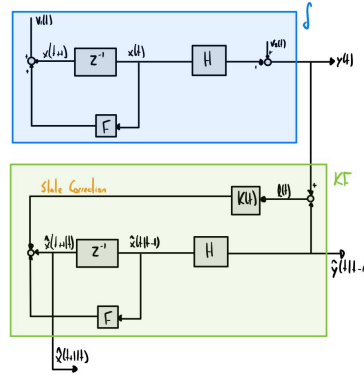


Figure 4: System and Kalman Filter Diagram

Basically, KF replicates the model of the system, plus a correction using a gain $K(t)$, which receives the output error.

The correction idea was known before KF, and was called state observer in feedback. Anyway, the fundamental contribution of Kalman is a closed expression for the optimal choice of the correction gain. $K(t)$ is not just a scalar number that can be found empirically, it is a matrix of size $n \times p$ that cannot be found easily.

The weak point of the KF is the knowledge of V_1, V_2, V_{12} . While the last two can be found, setting V_{12} as 0, and V_2 as the covariance error of the sensors, V_1 is harder to design.

Anyway, lately, it has been done optimization of V_1 and V_2 (or directly K) using ML. In this case, we need a dataset

3.5 Extensions

- **Multi-step Prediction:** Assume $\hat{x}(t+1|t)$ known. It can be proven that:

$$\hat{x}(t+k|t) = F^{k-1}\hat{x}(t+1|t)$$

$$\hat{y}(t+k|t) = H\hat{x}(t+k|t)$$

- **Filter $\hat{x}(t|t)$:** If F is invertible:

$$\hat{x}(t|t) = F^{-1}\hat{x}(t+1|t)$$

If F is not invertible, we can use a special version of KF formula, the “filter version”.

$$\begin{aligned}\hat{x}(t|t) &= F\hat{x}(t-1|t-1) + K_0(t)e(t) \\ \hat{y}(t|t-1) &= H\hat{x}(t|t-1) && \text{(Refers to prediction formulation)} \\ e(t) &= y(t) - \hat{y}(t|t-1) \\ K_0(t) &= (P(t)H^T)(HP(t)H^T + V_2)^{-1} \\ DRE &= \text{Identical to the prediction case.}\end{aligned}$$

This formula is valid if $V_{12} = 0$ only.

Notice that $K(t)$ and $K_0(t)$ are different. The first one is the gain of the prediction form, and the second one is the gain of the filter form. They are identical, but $K_0(t)$ has missing the F part

- **Exogenous Input $Gu(t)$:** In this case, we add to the system a $Gu(t)$, and it could be seen from “outside the box”.

We can add it to the KF formulation, reading it directly from the source, since it is known. Now, we see two inputs in the Kalman filter, $y(t)$ and $u(t)$.

In this case, $K(t)$ are exactly the same, but with $Gu(t)$. This is intuitive, since it does not add any uncertainty.

$P(t)$ changes as a consequence, but in terms of the formula, it remains the same.

- **Time-varying systems:** Let be the system exactly the same, but now F, G, and H depend on the time: $F = F(t), G = G(t), H = H(t)$. In this case, the kf equations remain exactly the same.

In this case, we say that the system is Linear Time Varying (LTV). Otherwise, it is called Linear Time Invariant (LTI).

3.6 Linear Time Variant

Note that even when the system is LTI, KF is on an “LTV system”, since the gain $K(t)$ is always LTV.

The LTV feature could rise 2 problems:

- Checking and guaranteeing the asymptotic stability of KF is very difficult. In the LTI case, we just have to check the eigenvalues of F. In the LTV case, even if all $\text{EIG}[F(t)]$ are inside the unitary circle, the stability is not guaranteed.
- Computational problems could arise. At every sampling time, we must update the computation of $K(t)$ and $P(t)$. It requires the inversion of a $p \times p$ matrix, which is costly.

Because of these problems, in real applications, in most of the cases, KF is used in its asymptotic version.

The basic idea is that DRE is an autonomous dynamic system: $P(t+1) = f(P(t))$. In this case,

$$\text{If } P(t) \rightarrow \bar{P} \quad \implies \quad K(t) \rightarrow \bar{K}$$

We can replace $K(t)$ with $\bar{K}$ and KF becomes an LTI system.

3.7 Stability and existence for convergent systems

Let's first assume $\bar{K}$ exists, check asymptotic stability of KF:

$$\begin{aligned} \hat{x}(t+1|t) &= F\hat{x}(t|t-1) + \underbrace{Gu(t)}_{\text{Does not affect stability}} + \bar{K}e(t) \\ &= F\hat{x}(t|t-1) + \bar{K}(y(t) - \hat{y}(t|t-1)) \\ &= F\hat{x}(t|t-1) + \bar{K}(y(t) - H\hat{x}(t|t-1)) \\ &= (F - \bar{K}H)\hat{x}(t|t-1) + \bar{K}y(t) \end{aligned}$$

Therefore, KF is asy. stable iff all the $EIG[(F - \bar{K}H)]$ are strictly inside the u.c.

Note that KF can be asy. stable also if the original system is unstable, thanks to $\bar{K}H$ contribution.

Since:

$$K(t) = (FP(t)H^T + V_{12})(HP(t)H^T + V_2)^{-1}$$

We can have a constant $\bar{K}$ if we have a constant $\bar{P}$.

We can have the solution of the discrete system, and the equilibrium point of DRE corresponds to the point where we apply the function and it remains the same:

$$\bar{P} = (F\bar{P}F^T + V_1) - (F\bar{P}H^T + V_{12})(H\bar{P}H^T + V_2)^{-1}(F\bar{P}H^T + V_{12})^T$$

This is a non-linear matrix, an algebraic equation named **Algebraic Riccati Equation (ARE)**. $\bar{P}$ is an equilibrium point of DRE if it is a solution of the corresponding ARE.

Now, there are 3 key questions:

- Existence: Does ARE have sdp solutions?
- Convergence: If existence, does $P(t)$ converges to $\bar{P}$. In other words, from every point the system will converge to the existent point?
- Stability: If existence and convergence, is the corresponding KF asymptotical stable? i.e., are all the $EIG[F - \bar{K}H]$ strictly inside u.c.?

The answer to these 3 questions is complicated, but there are two theorems that can provide sufficient conditions to satisfy the questions.

Theorem 3.1: Asymptotical KF theorem

If $V_{12} = 0$ and the system δ is asymptotical stable, then:

- ARE has one and only one sdp solution $\bar{P} \geq 0$
- DRE does converge to $\bar{P} \quad \forall P_0 \geq 0$
- The corresponding $\bar{K}$ is such that KF is asymptotic stable

For the second theorem, we must make a new definition.

Definition 3.2: Fully controllability

A system with two inputs, u and v i.e.:

$$x(t+1) = Fx(t) + Gu(t) + Lv(t)$$

is fully controllable from v if the system is controllable from the matrix Γ , i.e.:

$$\mathcal{R} = \begin{bmatrix} F & F\Gamma & F\Gamma^2 & \dots & F\Gamma^{n-1} \end{bmatrix} \text{ is full rank}$$

Where Γ is the matrix such that: $Lv(t) = \Gamma w(t)$, $w(t) \sim WN(0, I)$

Remark.

The meaning of this special controllability is the fact that noise $v_1(t)$ must affect all the states, we should not have clean or noise-free state equations.

Unintuitively, the noise is good.

Example.

For $x(t+1) = \frac{1}{2}x(t) + v_1(t)$, $v_1 \sim WN(0, 4)$ equation, an equivalent one is:

$$x(t+1) = \frac{1}{2}x(t) + 2w(t), \quad w \sim WN(0, 1)$$

So $\Gamma = 2$.

Theorem 3.3: Asymptotic KF theorem

If $V_{12} = 0$, (F, H) is fully observable, (F, Γ) is fully controllable, then:

- ARE has 1 dp solution $\bar{P} > 0$
- DRE does converge to $\bar{P} \quad \forall P_0 \geq 0$
- The corresponding $\bar{K}$ is such that KF is asymptotic stable

These two theorems are very useful. In practice, it allows us to skip the convergence analysis

of DRE directly.

The KF formulas can be applied under the restrictive assumption that both v_1 and v_2 are white noises. This is not true in a lot of real problems. For example, a sensor noise that is not WN, but colored.

This can be solved with a trick called model/system extension. Let see it with an example:

Example.

Let be the next model of order 1:

$$\delta : \begin{cases} x(t+1) = ax(t) + \eta(t) & \eta(t) = \frac{1}{1-cz^{-1}}e(t) & e(t) \sim WN(0,1) \\ y(t) = bx(t) + v_2(t), & v_2 \sim WN(0,1) \\ e(t) \perp v_2(t) \end{cases}$$

We can expand the noise model:

$$\eta(t+1) = c\eta(t) + e(t+1)$$

Define $v(t)=e(t+1)$:

$$\eta(t+1) = c\eta(t) + v(t), \quad v \sim WN(0,1) \quad , v \perp v_2$$

Now, set $x_1(t) := x(t)$ and $x_2(t) = \eta(t)$ and define the new system of order 2.

$$\delta : \begin{cases} x_1(t+1) = ax_1(t) + x_2(t) \\ x_2(t+1) = cx_2(t) + v(t) \\ y(t) = bx_1(t) + v_2(t) \end{cases}$$

And we can apply the KF formulas over this system. In this case:

$$F = \begin{bmatrix} a & 1 \\ 0 & c \end{bmatrix} \quad \wedge \quad H = \begin{bmatrix} b & 0 \end{bmatrix}$$

$$v_1(t) = \begin{bmatrix} 0 \\ v(t) \end{bmatrix} \implies v_1(t) \sim WN(0, V_1), V_1 = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

$$v_2(t) \sim WN(0,1), \quad V_2 = 1, \quad V_{12} = 0$$

3.8 Non-linear Systems

Consider a S.S model of a system with a non-linear dynamics:

$$\delta = \begin{cases} x(t+1) & = f(x(t), u(t)) + v_1(t) \\ y(t) & = h(x(t)) + v_2(t) \end{cases}$$

Where f and h are nonlinear functions (belonging to class C^1).

Example.

$$\delta = \begin{cases} x(t+1) &= \frac{1}{2}x^5(t) + u^3(t) + v_1(t) \\ y(t) &= e^{x(t)} + v_2(t) \end{cases}$$

This is called the Extended Kalman Filter (EKF). The diagram is exactly the same, but what changes is the Gain Function $K(t)$.

To design the feedback correction gain, we have 2 different options:

- The gain $K(t)$ itself becomes a nonlinear function
- The gain $K(t)$ remains as a linear time-varying block of the KF, but the output is $K(t) \cdot e(t)$ instead of applying K to $e(t)$.

In this case, $K(t)$ of EKF can be computed using the traditional formulas (see section 3.4).

Despite that, in this case we need to compute $F(t)$ and $H(t)$ at every sampling time:

$$F(t) = \left. \frac{\partial f(x(t), u(t))}{\partial x(t)} \right|_{x(t)=\hat{x}(t|t-1)} \quad \wedge \quad H(t) = \left. \frac{\partial h(x(t))}{\partial x(t)} \right|_{x(t)=\hat{x}(t|t-1)}$$

The procedure to apply EKF at time t is:

1. Take the last available prediction of the state $\hat{x}(t|t-1)$
2. Using $\hat{x}(t|t-1)$, compute the local linearization matrices $F(t)$ and $H(t)$
3. Compute $K(t)$ and update $P(t+1)$ using DRE
4. Compute $\hat{x}(t+1|t)$

Note that we are modeling a time-invariant nonlinear system as a time-variant linear system. Plus, as it is time-variant, the system does not have a steady state (asymptotic) solution.

Remark.

It is impossible to guarantee the asymptotical stability, so it needs an extensive empirical validation.

Remark.

Computational load is high, since at each time we must calculate $F(t), H(t), K(t), P(t)$

Today EKF is largely used, but with limitations for:

- Safety-Critical Applications.
- Mission-Critical Applications.

Example.

‘end-to-end’ application on a real-world problem: We need to estimate the unmeasurable variable speed $\dot{x}(t)$.

1. **Build the system model:** (Typically is a white box physical model)
Force balance (involving elastic and friction force):

$$\ddot{x}(t) \cdot M = -Kx(t) - C\dot{x}(t) + F(t)$$

This is a linear dynamical second-order system. It is an ordinary differential equation. Despite it is continuous time, we can transform it in standard state space form, with 2 first order equations.

Let x be the position and $\dot{x}$ the speed:

$$X(t) = \begin{bmatrix} x(t) \\ \dot{x}(t) \end{bmatrix} = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}$$

So the equations are:

$$\begin{cases} \dot{x}_1(t) &= x_2(t) \\ \dot{x}_2(t) &= -\frac{K}{M}x_1(t) - \frac{C}{M}x_2(t) + \frac{1}{M}F(t) \\ y(t) &= x_1(t) \end{cases}$$

Then:

$$A = \begin{bmatrix} 0 & 1 \\ -\frac{K}{M} & -\frac{C}{M} \end{bmatrix} \quad \wedge \quad B = \begin{bmatrix} 0 \\ \frac{1}{M} \end{bmatrix} \quad \wedge \quad \begin{bmatrix} 1 & 0 \end{bmatrix} \quad \wedge \quad D = 0$$

2. **Move from Continuous Time to Discrete Time:**

One of the simplest ways to discretize a CT system is to use the “Eulero-Forward” method:

$$\dot{x}(t) \approx \frac{x(t+1) - x(t)}{\Delta x}$$

Applying it to our system:

$$\begin{cases} \frac{x_1(t+1) - x_1(t)}{\Delta} &= x_2(t) \\ \frac{x_2(t+1) - x_2(t)}{\Delta} &= -\frac{K}{M}x_1(t) - \frac{C}{M}x_2(t) + \frac{1}{M}F(t) \\ y(t) &= x_1(t) \end{cases}$$

So isolating the variables:

$$\begin{cases} x_1(t+1) &= x_1(t) + \Delta x_2(t) \\ x_2(t+1) &= -\frac{K\Delta}{M}x_1(t) - \frac{C\Delta}{M}x_2(t) + \frac{\Delta}{M}F(t) + x_2(t) \\ y(t) &= x_1(t) \end{cases}$$

So, the standard SS form in discrete time is:

$$F = \begin{bmatrix} 1 & \Delta \\ -\frac{K\Delta}{M} & 1 - \frac{C\Delta}{M} \end{bmatrix} \quad \wedge \quad G = \begin{bmatrix} 0 \\ \frac{\Delta}{M} \end{bmatrix} \quad \wedge \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

3. Add the noises:

$$\begin{cases} x_1(t+1) &= x_1(t) + \Delta x_2(t) + v_{11}(t) \\ x_2(t+1) &= -\frac{K\Delta}{M}x_1(t) - \frac{C\Delta}{M}x_2(t) + \frac{\Delta}{M}F(t) + x_2(t) + v_{12}(t) \\ y(t) &= x_1(t) + v_2(t) \end{cases}$$

$$v_1(t) = \begin{bmatrix} v_{11}(t) \\ v_{12}(t) \end{bmatrix} \approx WN(0, V_1) \quad \wedge \quad v_2(t) \approx WN(0, V_2)$$

We have to find the values empirically:

- $V_{12} = 0$. (There's no reason for correlation)
- V_2 : It can be designed using the sensor dataset or by analyzing a signal recorded from the sensor
- V_1 : It is much more complicated. Usually, we can simplify the problem and assume a diagonal matrix:

$$V_1 = \begin{bmatrix} \lambda_1^2 & 0 \\ 0 & \lambda_2^2 \end{bmatrix}$$

Where λ 's are empirically tuned from data.

A new stream of research of KF is to estimate from data these meta parameters.

Remark.

In principle, KF is not an ML technique. In practice, today a lot of learning from data is used to tune the best meta-parameters.

4 Minimum Variance Control (MVC)

4.1 Introduction

We have completed the tasks related to the design of control systems: modeling the system, estimating unmeasurable variables, and designing the control algorithm.

The mathematical tools and framework we need for MVC are related to ARMAX models (see (Jeon, 2025)).

We can see MVC as a general tool of optimization of stochastic feedback on dynamical systems. It can be diagramed as follows:

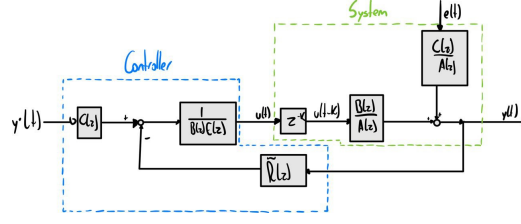


Figure 5: System and MVC system

4.2 The problem

The MVC problem is set around ARMAX models of I/O systems (In canonical representation):

$$y(t) = \frac{B(z)}{A(z)} u(t-k) + \frac{C(z)}{A(z)} e(t) \quad e \sim WN(0, \ell^2)$$

Where:

$$B(z) = b_0 + b_1 z^{-1} + \dots + b_p z^{-p}$$

$$A(z) = 1 + a_1 z^{-1} + \dots + a_n z^{-n}$$

$$C(z) = 1 + c_1 z^{-1} + \dots + c_m z^{-m}$$

The problem is built on the assumption that $\frac{B(z)}{A(z)}$ is minimum phase, i.e., all the zeros of $B(z)$ (zeros) are strictly inside the u.c.

Remark.

When there's a jump on the input, a step, if the value of $y(t)$ is minimum phase, then the response always goes in the same direction of the final steady-state value.

MVC cannot be applied to a non-minimum phase system. In this case, we need generalized MVC

The classical goal of Control System is to let $y(t)$ follow the target $y^0(t)$ (the desired output). In a formal way, we need to find an input signal $u(t)$ that minimizes the following performance index:

$$J = E[(y(t) - y^0(t))^2]$$

i.e. the variance of the tracking error. This is why it is called the minimum variance error.

To solve this problem, we need to make some additional assumptions on signal $y^0(t)$:

- $y^0(t)$ and $e(t)$ are uncorrelated: $y^0(t) \perp e(t)$
- $y^0(t)$ is unpredictable, we have no preview of the future desired output. Then, at present time t , the best prediction is simply $y^0(t)$:

$$\hat{y}^0(t+k|t) = y^0(t)$$

Finally, summarizing, the general setup of the problem is:

$$\delta : y(t) = \frac{B(z)}{A(z)}u(t-k) + \frac{C(z)}{A(z)}e(t) \quad e \sim N(0, \ell^2)$$

where:

- It is minimal phase: $B(z)$ has all the roots inside the u.c.
- $C(z)$ and $A(z)$ are in canonical form
- $y^0(t) \perp e(t)$
- $y^0(t)$ is unpredictable, so $\hat{y}^0(t+k|t) = y^0(t)$
- The goal is to minimize $J = E[(y(t) - \hat{y}(t))^2]$

Proof of the MVC problem.

The main trick is to split $y(t)$ into the predictor and prediction error:

$$\epsilon(t) = y(t) - \hat{y}(t|t-k) \iff y(t) = \hat{y}(t|t-k) + \epsilon(t)$$

So:

$$\begin{aligned} J &= E[(\hat{y}(t|t-k) + \epsilon(t) - y^0(t))^2] = E[(\hat{y}(t|t-k) - y^0(t) + \epsilon(t))^2] \\ &= E[(\hat{y}(t|t-k) - y^0(t))^2] + E[\epsilon(t)^2] + \underbrace{2E[(\hat{y}(t|t-k) - y^0(t)) \cdot \epsilon(t)]}_{0 \text{ by constriction.}} \end{aligned}$$

Si minimizing J wrt u is equivalent to minimizing:

$$E[(\hat{y}(t|t-k) - y^0(t))^2] + E[\epsilon(t)^2]$$

with respect to u .

4.3 Optimal Predictor

- Make a k -steps polynomial long division between $C(z)$ and $A(z)$.
- Solution is $E(z)$, the residual is $R(z) = \tilde{R}(z) \cdot z^{-k}$
- It can be rewritten:

$$\frac{C(z)}{A(z)} = E(z) + \frac{\tilde{R}(z)z^{-k}}{A(z)}$$

- The formula for the k -steps ahead prediction of ARMAX:

$$\hat{y}(t|t-k) = \frac{B(z)E(z)}{C(z)}u(t-d) + \frac{\tilde{R}(z)}{C(z)}y(t-k)$$

We take $d=k$, (see additional section 4.6)

- Now as we can't use the identity:

$$\hat{y}(t+k|t) = y^0(t+k)$$

Since $y^0(t+k)$ is not available at time t , the best we can do is use the identity $\hat{y}(t+k|t) = y^0(t)$ (see section 4.5), and plug it into the formula:

$$y^0(t) = \frac{B(z)E(z)}{C(z)}u(t) + \frac{\tilde{R}(z)}{C(z)}y(t)$$

And isolating u , we have the general formula of MVC:

$$u(t) = \frac{1}{B(t)E(t)}(C(t)y^0(t) - \tilde{R}(z)y(t))$$

4.4 Stability

To check the stability of a feedback system, we must compute the loop function:

$$L(z) = F_1(z) \cdot F_2(z)$$

where:

- $F_1(z)$ is the controller-system part of the loop. In this case:

$$F_1(z) = \frac{1}{B(z)E(z)} \frac{B(z)}{A(z)} z^{-n}$$

- $F_2(z)$ is the system-controller part of the loop. In this case:

$$F_2(z) = \tilde{R}(z)$$

Build the so called characteristic polynomial:

$$\chi(z) = L_N(z) \overset{(-)}{+} L_D(z) \quad (\text{Numerator and denominator})$$

Where it is a $+$ if the F_2 is a $-$ feedback, and it is a $-$ if F_2 is a positive feedback.

The feedback system is asymptotically stable if all the roots of $\chi(z)$ are strictly inside the u.c.

Applied to this case:

$$L(z) = \frac{1}{B(z)E(z)} \frac{B(z)}{A(z)} z^{-n} \cdot \tilde{R}(z)$$

So:

$$\chi(z) = L_N(z) + L_D(z) = B(t)\tilde{R}(z)z^{-n} + B(z)E(z)A(z) = B(z)(\tilde{R}z^{-k} + E(z)A(z)) = B(z)C(z)$$

All roots of $B(z)$ are strictly inside the u.c., thanks to the min phase assumption. Plus, all the roots of $C(z)$ are strictly inside the u.c., due to the canonical form assumption on $C(z)$ and $A(z)$.

Then, the MVC control system is guaranteed to be always asymptotically stable.

4.5 y estimation

Previously, we assumed the best estimate of $y(t)$ is $y^0(t-k)$; in this section, we explain this. Note that in a closed loop, the output $y(t)$ can be written as:

$$y(t) = F_{y^0y}(z) \cdot y^0(t) + F_{ey}(z) \cdot e(t)$$

In a feedback system there is $F_1(z)$ and $F_2(z)$. The first one is between in and out, and the second one is the feedback of out in the in. In this case, the TF from in to out is:

$$\frac{F_1(z)}{1 + \overset{(-)}{F_1(z)F_2(z)}}$$

Applying to the system:

$$F_{y^0y}(z) = \frac{C(z) \frac{1}{B(z)E(z)} \frac{B(z)}{A(z)} z^{-k}}{1 + \tilde{R}(z) \frac{1}{E(z)B(z)} \frac{B(z)}{A(z)} z^{-k}} = \dots = z^{-k}$$

On the other hand, from $e(t)$ to $y(t)$:

$$F_{ey}(z) = \frac{\frac{C(z)}{A(z)}}{1 + \tilde{R}(z) \frac{1}{E(z)B(z)} \frac{B(z)}{A(z)} z^{-k}} = \dots = E(z)$$

The closed loop system has a very simple behavior:

$$y(t) = y^0(t-k) + E(z)e(t)$$

Although the ideal behavior would be:

$$y(t) = y^0(t-k) + 0$$

Therefore, MVC is optimal. It is not possible to find a better minimizer.

4.6 Choosing predictor for MVC

To obtain the MVC, we need to use a predictor:

$$\hat{y}(t|t-k) = \frac{B(z)E(z)}{C(z)} u(t-d) + \frac{\tilde{R}(z)}{C(z)} y(t-k)$$

Since the predictor depends on the $t-k$ steps, we need to constrain:

$$t-k \geq t-d \quad \implies \quad d \geq k$$

On the other hand, shifting the predictor by d steps and estimating it by $y^0(t)$:

$$\hat{y}(t+d|t-k+d) = \frac{B(z)E(z)}{C(z)}u(t) + \frac{\tilde{R}(z)}{C(z)}y(t-k+d) = y^0(t)$$

And isolating $u(t)$:

$$u(t) = \frac{1}{B(z)E(z)} \left[C(z)y^0(t) - \tilde{R}(z)y(t-k+d) \right]$$

And to know what is u at time t , we need to constrain:

$$t \geq t - k + d \quad \implies \quad k \geq d$$

So, taking both requirements, we need:

$$d = k$$

4.7 Generalized MVC (GMVC)

There are 3 main limits of MVC:

- It can be applied only to min-phase systems
- We cannot moderate the effort/intensity action $u(t)$.
- We cannot design a specific behavior between $y^0(r)$ and $y(t)$.

To overcome these limits, an extension of MVC can be used, the Generalized Minimum Variance Control (GMVC). GMVC calculates:

$$J = E \left[(y(z) - P(z)y^0(t) + Q(z)u(t))^2 \right]$$

It has some advantages:

- GMVC can manage non-minimum phase systems. The trick is to put in $P(z)$ the non-min phase zeros of $B(z)$.
- It has a new term, the third one, which penalizes the control effort.
- We can introduce a reference model $P(z)$, that indicates the desired $y^0(t) - y(t)$ behavior

The reference model $P(z)$. In general, in a control feedback system, the expected result is “Perfect Tracking”, in this case, the model is $P(z)$.

Anyway, there are cases where $P(z) = 1$ is not the best closed-loop behavior. For example, sometimes, we want a slow behavior in the desired behavior. In this specific case, a good reference model could be a Low-pass filter.

4.8 Discretization of Analog Dynamical Systems

Let be a very general control system layout. It is a physical system to be controlled (analog), monitored, predicted, etc. It receives an input signal $u(t)$ and it gives an output signal $y(t)$. On the other hand, the control system (digital) is created. We have in both senses a Digital-Analog (D/A) and Analog-Digital (A/D) converters that connect the physical and the control system.

- Specifically, the A/D converter makes a discretization of both time and amplitude:
 - ΔT is the time discretization
 - The amplitude discretization is the number of available discretization levels. For example, for 10 bits, there are 1024 levels.

The quality and cost of an A/D converter depend on both levels of discretization.

- On the other hand, the D/A converter is typically called ‘Holder’. It outputs a step function, given by the last signal received and its respective output. Specifically, this is called “Zero-order-hold”. Higher hold levels can be used, but in practice, most of the real D/As are zero-order.

Summarizing, there is a discrete time system that is related with the digital system by $u(t)$ and $y(t)$. There are two different ways to model this:

- Make an experiment: Collect DT data $\{u(1), u(2), \dots, u(N)\}, \{y(1), y(2), \dots, y(N)\}$ and then estimate a dynamical model (typically black-box) directly from data (learning from data).
- Convert the continuous time model (State-Space) into a Discrete Time model.

4.8.1 State-Space Discretization

The most used discretization method is State-Space transformation.

Given a system:

$$\delta : \begin{cases} \dot{x} = Ax + Bu \\ y = Cx + Du \end{cases}$$

And a time sampling ΔT . The system can be transformed to a discrete time system:

$$\delta : \begin{cases} x(t+1) = Fx(t) + Gu(t) \\ y(t) = Hx(t) + Du(t) \end{cases}$$

The transformation is done by the following formulas:

- $F = e^{A\Delta T}$
- $G = \int_0^{\Delta T} e^{As} B ds$
- $H = C$
- $D = D$

The characteristics of these systems are related as follows:

- The eigenvalues of A are transformed following the sampling transformation rule:

$$\ell_F = e^{\ell_A \cdot \Delta T}$$

Where ℓ_F is an eigenvalue of F and ℓ_A of A.

- On the other hand, the system in the continuous time domain (S) is stable if the real part is negative. The system in the discrete time (Z) is stable if the eigenvalues are in the unitary circle in the complex plane.
- In the continuous setting, the transfer function has h zeros in the numerator and n in the denominator (poles), where $h < n$ if the system is strictly proper. In the discrete setting, the denominator has n zeros (poles) and the numerator n-1 zeros. There are n-h-1 zeros generated by the discretization step, called “Hidden Zeroes”. Those generated zeros does not follow a simple rule, and usually some of them are outside u.c., making the system non-min phase

4.8.2 Derivative Discretization

Other category of discretizations frequently used are the derivative discretization:

- **Eulero Backward:**

$$\dot{x} \approx \frac{x(t) - x(t-1)}{\Delta T} = \frac{x(t) - z^{-1}x(t)}{\Delta T} = \frac{z-1}{z\Delta T}x(t)$$

- **Eulero Forward:**

$$\dot{x} \approx \frac{x(t+1) - x(t)}{\Delta T} = \frac{zx(t) - x(t)}{\Delta T} = \frac{z-1}{\Delta T}x(t)$$

- **Combination of backward and forward**

$$\dot{x} \approx \left[\frac{z-1}{\Delta T} \cdot \frac{1}{\alpha z + (1-\alpha)} \right] x(t) \quad 0 \leq \alpha \leq 1$$

A very used case is the Tustin method, with $\alpha = \frac{1}{2}$

4.8.3 Choosing ΔT

Any signal (like a sound wave, electrical signal, etc.) can be seen as a sum of sinusoids (sine and cosine waves) of different frequencies.

Definition 4.1: Maximum Frequency

The Maximum Frequency of a signal ($f_{\max}$) is the highest frequency component present in a signal (The highest frequency of a sinusoid function summing in the signal).

Let ΔT be the sampling time of S, define:

Definition 4.2: Sampling Frequency

Given a signal, the sampling frequency is the frequency at which we sample out of the signal:

$$f_s = \frac{1}{\Delta T} \text{Hz (if } \Delta T \text{ in seconds)}$$

The Sampling angular frequency is $\omega_s = \frac{2\pi}{\Delta T} = 2\pi f_s$ (rad/s).

Definition 4.3: Nyquist Frequency

The Nyquist Frequency f_N and Nyquist angular frequency ω_N are half of the sampling frequency and angular sampling frequency respectively:

$$f_N = \frac{1}{2} f_s \text{ Hz} \quad \omega_N = \frac{1}{2} \omega_s \text{ (rad/s)}$$

Definition 4.4: Aliasing

Aliasing is a phenomenon in signal processing where different continuous-time frequencies become indistinguishable (or “aliases” of each other) when a signal is sampled below a certain rate.

Theorem 4.5: Shannon Theorem

The signal does not alias if $f_N \geq f_{\max}$. i.e., from f_N the frequencies start to aliasing

A general intuitive rule to select f_s is “The bigger the better”, since the approximations become worse as the frequencies are closer to f_N . Therefore, as f_s (or ω_s) increases, the wider is the bandwidth of a good approximation of $G(z)$ with $G(s)$.

Anyway, there are some problems if f_s is too big:

- Sampling devices (A/D and D/A) become more expensive
- Computational cost: Since we need to update the algorithm more often
- Cost of memory: Buffering a signal more often needs more memory
- Numerical Precision: If ΔT is very small, the poles of DT are very close to 1 (Remember $\ell_F = e^{\ell_A \cdot \Delta T}$)

Therefore, the rule of thumb is $f_N \approx 10$ times the expected control system bandwidth of interest.

5 Frequency Domain Model Identification

5.1 Introduction

In this section, we will review black-box system model identification using a classic parametric approach for I/O (T.F.) models in the frequency domain.

This method is very used in closed loop control applications. Classically, it consists of 4 steps of M.L. parametric methods:

1. Experiment Design / Data Collection / Data pre-processing
2. Selection of a family of parametric models ($m(\theta)$, θ a parametric vector)
3. Definition of a parametric index $J(\theta)$
4. Optimization: $\hat{\theta} = \arg \min_{\theta} \{J(\theta)\}$ to get the optimal model $m(\hat{\theta})$

We will focus on steps 1 and 2, that are special in the frequency domain system identification.

5.2 Method Idea

1. Make a set of a “single sinusoid” (or single frequency or single tone) excitation experiment.
2. For each experiment: Extract and estimate a single point of the frequency response of the system.
3. Fit the measured frequency response with the FR of the model (Supervised learning in frequency domain).

5.3 System Excitation

Although in some cases we have the data instead of the model itself, in a lot of cases we have the systems, and we need to design which experiments we will conduct to obtain the data to train the model. The data feeding of the model is called model excitation. The right way to excite a system can differ from system to system, but there are two main signals used for identification:

- **Sinesweep:** Multiple Sine Experiments. We can span all the frequency ranges important for identifying the system. The dataset contains useful information in both time domain and frequency domain.
- **Pseudo-Random Binary Signal (PRBS):** It is a signal which only has two values $+A$ and $-A$, and commute between them at “random”. From the spectral domain, PRBS has a constant spectrum up to its cutting frequency, so it contains a lot of information in frequency domain.

Usually, the optimization is performed in one dataset and then validated in a qualitative different dataset, so it avoids overfitting.

Example.

I/O dynamics estimation of a steer to yaw rate (How the wheels turn based on the movement of the handle in a car). This I/O is very useful to design electronic stability control (ESC, ESP), and it can help to build autonomous cars (Lateral dynamics control).

In the experiment design, the first step is to select a set of excitation frequencies: $\omega_1, \dots, \omega_H$, separated by $\Delta\omega$, a fixed constant distance between two consecutive ω s.

The next step is to perform H independent I/O experiments on the system, taking the signals of each ω , modeling:

$$u_i(t) = A_i \sin(\omega_i t) \xrightarrow{\text{System}_i} y_i$$

For each of these experiments, we need to have N data points.

Remark.

The amplitude of the H sinusoids ($A_1, \dots, A_H$) can be identical, but usually, they have decreasing values to meet the actuation limit (e.g. power limitation of a steer actuator)

Example.

Following the previous example: Given an input $S(t)$ in a car, the requested steering torque is proportional to S: $T(t) = K \cdot S(t)$.

Steering power requested: torque $\times$ rotational speed:

$$power = K \cdot S(t) \cdot \dot{S}(t)$$

Assume that $S(t) = A_i \sin(\omega_i t)$, then, the steer power is:

$$K A_i \sin(\omega_i t) \cdot \omega A_i \cos(\omega_i t) = \underbrace{K A_i^2 \omega_i}_{\text{Amplitude power signal}} \sin(\omega_i t) \cos(\omega_i t)$$

So, if there is a power limitation:

$$K A_i^2 \omega_i = constant \implies A_i = \sqrt{\frac{constant}{K \cdot \omega_i}}$$

Now, we can focus on the generic ith experiment.

We are assuming that the systems are LTI. For LTI systems, if the input is a sinusoid at frequency ω_i , the output is a sinusoid at frequency ω_i as well (But the amplitude and phase can change).

5.4 Denoised Measures

Anyway, in real applications, $y_i(t)$ is never exactly a perfect sinusoid. This, due to output, measure, internal, and other noises. Also, due to small non-linear effects (that we neglect).

In order to analyze, we look for an exact linear time-invariant (LTI) approximation. We call $\hat{y}_i(t)$ to the noise-cleaned version of $y_i(t)$. The preprocessing problem to find $\hat{y}_i(t)$ is a simple model identification problem.

The dataset is:

$$\{y_i(1), \dots, y_i(N)\}$$

With a model $y_i(t)$ that can be:

$$\hat{y}_i(t) = \begin{cases} B_i \sin(\omega_i t + \varphi_i) \\ a_i \sin(\omega_i t) + b_i \cos(\omega_i t) \end{cases}$$

These two models are equivalent, but we will choose the second one since it is linear wrt the parameters. Now, we can estimate a_i and b_i using a classic parametric supervised learning, with the performance index:

$$J_N(a_i, b_i) = \frac{1}{N} \sum_{t=1}^N [y_i(t) - (a_i \sin(\omega_i t) + b_i \cos(\omega_i t))]^2$$

Now the optimization is easy, since this is a convex function. We can simply take the first order conditions:

$$\begin{cases} \frac{\partial J_N(a_i, b_i)}{\partial a_i} = 0 \\ \frac{\partial J_N(a_i, b_i)}{\partial b_i} = 0 \end{cases} \implies \begin{cases} \sum_{t=1}^N \sin(\omega_i t) \cdot [y_i(t) - (a_i \sin(\omega_i t) + b_i \cos(\omega_i t))] = 0 \\ \sum_{t=1}^N \cos(\omega_i t) \cdot [y_i(t) - (a_i \sin(\omega_i t) + b_i \cos(\omega_i t))] = 0 \end{cases}$$

Then, it reduces to solving:

$$\begin{bmatrix} \sum_{t=1}^N \sin^2(\omega_i t) & \sum_{t=1}^N \sin(\omega_i t) \cos(\omega_i t) \\ \sum_{t=1}^N \sin(\omega_i t) \cos(\omega_i t) & \sum_{t=1}^N \cos^2(\omega_i t) \end{bmatrix} \cdot \begin{bmatrix} a_i \\ b_i \end{bmatrix} = \begin{bmatrix} \sum_{t=1}^N y_i(t) \sin(\omega_i t) \\ \sum_{t=1}^N y_i(t) \cos(\omega_i t) \end{bmatrix}$$

And by classic OLS, the optimum is:

$$\begin{bmatrix} \sum_{t=1}^N \sin^2(\omega_i t) & \sum_{t=1}^N \sin(\omega_i t) \cos(\omega_i t) \\ \sum_{t=1}^N \sin(\omega_i t) \cos(\omega_i t) & \sum_{t=1}^N \cos^2(\omega_i t) \end{bmatrix}^{-1} \begin{bmatrix} \sum_{t=1}^N y_i(t) \sin(\omega_i t) \\ \sum_{t=1}^N y_i(t) \cos(\omega_i t) \end{bmatrix}$$

Now, we move back to an amplitude phase representation:

$$\hat{y}_i(t) = \hat{a}_i \sin(\omega_i t) + \hat{b}_i \cos(\omega_i t) = \hat{B}_i \sin(\omega_i t + \hat{\varphi}_i) = \hat{B}_i \sin(\omega_i t) \cos(\hat{\varphi}_i) + \hat{B}_i \cos(\omega_i t) \sin(\hat{\varphi}_i)$$

And now, to find these last parameters that correspond to the ones found before:

$$\begin{cases} \hat{B}_i \cos(\hat{\varphi}_i) = \hat{a}_i \\ \hat{B}_i \sin(\hat{\varphi}_i) = \hat{b}_i \end{cases} \implies \frac{\hat{a}_i}{\hat{b}_i} = \frac{\cos(\hat{\varphi}_i)}{\sin(\hat{\varphi}_i)} \implies \hat{\varphi}_i = \arctan\left(\frac{\hat{b}_i}{\hat{a}_i}\right)$$

$$\Rightarrow \hat{B}_i = \frac{\frac{\hat{a}_i}{\cos(\varphi_i)} + \frac{\hat{b}_i}{\sin(\varphi_i)}}{2}$$

So:

$$\{\hat{a}_i, \hat{b}_i\} \iff \{\hat{B}_i, \hat{\varphi}_i\}$$

And as a result, a system:

$$A_i \sin(\omega_i t) \longrightarrow y_i(t) = \hat{y}_i(t) = \hat{B}_i \sin(\omega_i t + \hat{\varphi}_i)$$

Then, we call amplitude gain and phase shift to, respectively:

$$\frac{\hat{B}_i}{A_i} \quad \wedge \quad \hat{\varphi}_i - 0 = \hat{\varphi}_i$$

So the frequency response of the system at frequency ω_i is:

$$\frac{\hat{B}_i}{A_i} e^{j\hat{\varphi}_i}$$

Finally, by repeating this procedure H times, we have H estimated values of the frequency response of the system:

$$\left\{ \frac{\hat{B}_1}{A_1} e^{j\hat{\varphi}_1}, \frac{\hat{B}_2}{A_2} e^{j\hat{\varphi}_2}, \dots, \frac{\hat{B}_H}{A_H} e^{j\hat{\varphi}_H} \right\}$$

5.5 Performance Index Definition

Since we are now estimating a signal that is represented in different frequencies, we need a new performance index to identify the model in the frequency domain. We build:

$$J_H(\Theta) = \frac{1}{H} \sum_{i=1}^H \left(\left| W(e^{j\omega_i}, \theta) - \frac{\hat{B}_i}{A_i} e^{j\omega_i} \right|^2 \right)$$

Where we compare the modeled frequency response with the measured (denoised) frequency response.

Now we optimize this index:

$$\hat{\Theta}_H = \arg \min_{\theta} \{J_H(\theta)\}$$

This is usually a non-quadratic optimization problem, so it is solved by standard tools of iterative optimization (gradient, newton, quasi-newton, etc).

Finally, the estimated model is:

$$m(\hat{\Theta}_H) : \quad \xrightarrow{u(t)} \quad \hat{W}(z; \hat{\Theta}_H) \quad \xrightarrow{y(t)}$$

5.6 Additional Remarks

- To choose a ω_H , we need to choose a meaningful Bandwidth (BW) for the model. It is usually taken a 3 times frequency the control one ($\omega_{\max}$ or the frequency I want to model instead).

- In some cases, we may want to have higher modeling precision for specific BWs in $\{\omega_1, \dots, \omega_H\}$. This can be modeled by using non-uniform weights γ_i in the performance index, having a performance index:

$$J_H(\Theta) = \frac{1}{H} \sum_{i=1}^H \left(\gamma_i \left| W(e^{j\omega_i}; \theta) - \frac{\hat{B}_i}{A_i} e^{j\omega_i} \right|^2 \right)$$

A similar focus effect can be obtained by designing from the beginning an oversampling of the focused part of the BW. In other words, we can put loads of ω_i 's close to the frequencies we want to be focused on.

- If we want to collect data with an I/O experiment of an unstable system, the open-loop experiment cannot be performed since the system is not stable. In these situations, a very useful trick is to design a simple stabilizing controller and perform a closed-loop experiment over it.

6 Software Sensing (SW) with B.B methods

6.1 Introduction

In section 3 we have seen SW sensing with Kalman Filter, a model-based method with a correction feedback. Specifically:

- In this kind of model, a white-box class of model is needed.
- In principle, a training dataset is not needed (even if we need it to find V_1 and V_2 empirically).
- It is a constructive method based on a feedback scheme.
- In principle, can totally estimate unmeasurable signals (states)

In this case, we want to solve the same SW sensing problem in a B.B. way. It is possible to do this, but the state $x(t)$ must be measured at the training stage.

Now, the starting point is:

$$\{u(1), u(2), \dots, u(N)\}$$

$$\{y(1), y(2), \dots, y(N)\}$$

$$\{x(1), x(2), \dots, x(N)\}$$

We want to use $u(t)$ and $y(t)$ as inputs to estimate x ($\hat{x}(t)$).

Now, if we do it by a parametric model, we need to estimate a mathematical model with the following architecture:

$$x(t|t) = W_{ux}(z; \theta) \cdot u(t) + W_{yx}(z; \theta) \cdot y(t)$$

The classical parametric approach is to find the performance index:

$$J_N(\Theta) = \frac{1}{N} \sum_{t=1}^N (x(t) - (W_{ux}(z; \theta) \cdot u(t) + W_{yx}(z; \theta) \cdot y(t)))^2$$

And optimize it:

$$\hat{\Theta}_H = \arg \min_{\theta} \{J_N(\Theta)\} \quad \implies \quad \hat{x}(t|t) = W_{ux}(z; \hat{\theta}) \cdot u(t) + W_{yx}(z; \hat{\theta}) \cdot y(t)$$

Once the SW sensor has been estimated, we can remove the physical sensors of the state $x(t)$ and replace it with the expression above.

The above system identification can be done with many different system identification techniques (e.g. 4SID).

When the system is non-linear and dynamical, we can use the same idea, but we need to design non-linear models. There are a lot of architectures for these kind of sensors, we will see some of them in this section.

6.2 Recurrent (Dynamical) Neural Networks

It consists of a neural network architecture with recurrent connections (see section 23 at (Jeon, 2024)). This architecture is the most general and most intuitive solution. Anyway, it is not much used since it requires a lot of computational resources, and it is difficult to guarantee the stability of the solution.

6.3 FIR architecture

A second type of architecture consists of splitting the sw sensor into a dynamical linear part and a static non-linear part. This is done with a non-recursive FIR scheme.

First, we use as a dynamical linear system a fixed number of z^{-k} s operators, in order to obtain past realizations, in both $u(t)$ and $y(t)$. With this, we can give:

$$\{u(t-1), u(t-2), \dots, u(t-n_u), y(t), y(t-1), y(t-2), \dots, y(t-n_y)\}$$

to a BB static model, for example, an static NN.

This architecture have is stable by construction (FIR like architecture). The problem is the high dimension of the static model. As:

$$u(t) \in \mathbb{R}^m \quad \wedge \quad y(t) \in \mathbb{R}^p \quad \wedge \quad x(t) \in \mathbb{R}^n$$

The static model will be:

$$f(\cdot, \Theta) : \mathbb{R}^{(m \times n_u) + (p \times (n_y + 1))} \rightarrow \mathbb{R}^n$$

6.4 IIR architecture

It consists of splitting the software sensor with a recursive feedback. It consists on the same kind of architecture as section 6.3, but adding a feedback simply with the backward operator, i.e.:

$$\{x(t-1), x(t-2) \dots, x(t-n_x)\}$$

The advantage of this model over the FIR one is that n_u and n_y can be much smaller, since part of ‘the memory’ is contained in the delayed x’s.

The disadvantage is that after training $x(t)$ must be replaced with a feedback of $\hat{x}(t|\theta)$. This could introduce an unstable behavior into the model.

6.5 Meta Architecture

An extra architecture that can be added to the rest ones is an architecture we add when we have a-priori knowledge of the system. It basically consists of adding a pre-processing stage, using previous knowledge of the system, achieving to reduce the number of components of the signals from $u(t)$ and $y(t)$ to a smaller features:

$$R_1(t), R_2(t), \dots, R_H(t)$$

The disadvantage of this architecture is that the preprocessing block needs an expert capable of reducing the number of signals.

The advantage is that it has cheaper models, and it is usually a much simpler model.

6.6 Model Based vs Black-box SW sensing

Characteristic	KF (model-based)	BB
Need of a physical model	Yes	No
Need of a training dataset	No, but tuning variances needed	Yes
Interpretability of solution	Yes	No
Ease for retuning for a similar model	Yes	No
Accuracy/Quality of SW sensor	Good	Very Good
Capability of measuring unmeasurable states	Yes	No

Table 1: Kalman Filter vs Black Box Models comparison

7 Grey Box (GB) System Identification

7.1 Introduction

The gray-box model is a model that is the middle point between white and black box. Gray Box Models is both prior-knowledge and data driven, but having a mix that allows it not to depend completely on one of them.

In this case, we have a model class $M(\Theta)$ that models the process, but that needs some data-dependent parameters.

A model built in a gray box way can be modeled as follows:

$$\delta : \begin{cases} x(t+1) &= f(x(t), u(t); \Theta) + v_1(t) \\ y(t) &= h(x(t); \Theta) + v_2(t) \end{cases}$$

where $f(\cdot)$ and $h(\cdot)$ are known functions. These functions can have some (a few) unknown parameters Θ that are estimated by the dataset:

$$\{u(1), u(2), \dots, u(N)\}$$

$$\{y(1), y(2), \dots, y(N)\}$$

KF is a method for white model SW-sensing, but it can be re-used also for Gray Box Model Identification.

7.2 Kalman Filter with State Extension

We can solve this problem using the Kalman Filter, doing a “State extension”:

$$\begin{cases} x(t+1) &= f(x(t); u(t); \Theta(t)) + v_1(t) \\ \Theta(t+1) &= \Theta(t) + v_\Theta(t) \\ y(t) &= h(x(t); \Theta(t)) + v_2(t) \end{cases}$$

Therefore, from now on, we consider the extended state vector as:

$$X_E(t) = \begin{bmatrix} x(t) \\ \Theta(t) \end{bmatrix}$$

The core dynamical relationship:

$$\Theta(t+1) = \Theta(t)$$

It is the equation of a constant quantity. This has its eigenvalue in 1, so it is not asy. stable. This is no problem, since KF can deal with stable and unstable systems.

The noise $v_\Theta(t)$ is added to model how much we trust the initial condition. We assume on it the usual assumptions:

$$v_\Theta(t) \sim WN(0, V_\Theta) \quad \wedge \quad v_1 \perp v_\Theta \quad \wedge \quad v_2 \perp v_\Theta$$

And a typical simplification structure is to take $V_\Theta = \ell_\Theta^2 I$, a diagonal matrix. The parameter ℓ_Θ^2 is empirically tuned. When larger, it allows faster convergence with a higher variance at the steady state. When it is shorter, it converges more slowly, but the steady state has a lower variance.

Remark.

- The method is very useful, specially if it is used at runtime, because Θ is time varying.
- KF provides at the same time $\hat{x}(t|t)$ and $\hat{\Theta}(t|t)$
- Be careful with the ratio between sensors and estimation amount, since it means condensing too much information into very specific values. Ex: 3/25 may be too much, but 3/7 should be ok.

7.3 Optimization

The gray box methods can be optimized by diverse methods. The most used one is the **Simulation Error Method (SEM)**. It consists of doing a simulation of the model and check the difference between $y_{sim}(t, \theta)$ and the real output.

Mathematically, the performance index of SEM is:

$$J_N(\Theta) = \sum_{t=1}^N (y(t) - m(\Theta) \cdot u(t))^2$$

References

- Jeon, M. S. (2024). *Artificial neural networks and deep learning*. Retrieved from https://github.com/minsoos/class_notes/tree/main/ANN (Accessed: 2025-02-25)
- Jeon, M. S. (2025). *Model identification and data analysis 1*. Retrieved from https://github.com/minsoos/class_notes/tree/main/MIDA1 (Accessed: 2025-04-14)
- Savaresi, S. (2025). *Model identification and data analysis - 2nd module*. Retrieved from https://www11.ceda.polimi.it/schedaincarico/schedaincarico/controller/scheda_pubblica/SchedaPublic.do?&evn_default=evento&c_classe=838109&_pj0=0&_pj1=3fe685c7a4cb09decfd0f15ec527dbea (Accessed: 2025-04-14)